Київський національний університет ім. Тараса Шевченка

Кафедра мережевих та інтернет технологій

**Лабораторна робота № 1**

**Дисципліна: Системи програмування**

**Тема:** Потоки. Клас thread. Пріоритети потоків

Виконав: Студент групи МІТ-41

Пугач Назар

## Хід роботи

**Завдання 1:** Реалізуйте програму на C#, яка створює два потоки: Перший потік виводить числа від 1 до 40 з паузою 200 мс. Другий потік виводить літери від A до Z з паузою 300 мс. Використайте методи Start(), Sleep() та властивість Name.

```
using System;
using System.Text;
using System.Threading;

class Program
{
    static void Main()
    {
        // Створення першого потоку (числа)
        Thread numbersThread = new Thread(PrintNumbers);
        numbersThread.Name = "NumbersThread";

        // Створення другого потоку (літери)
        Thread lettersThread = new Thread(PrintLetters);
        lettersThread.Name = "LettersThread";

        // Запуск потоків
        numbersThread.Start();
        lettersThread.Start();

        // Очікування завершення потоків
        numbersThread.Join();
        lettersThread.Join();

        Console.WriteLine("Thread execution has completed.");
    }

    static void PrintNumbers()
    {
        for (int i = 1; i <= 40; i++)
        {
            Console.WriteLine($"{Thread.CurrentThread.Name}: {i}");
            Thread.Sleep(200);
        }
    }

    static void PrintLetters()
    {
        for (char c = 'A'; c <= 'Z'; c++)
        {
            Console.WriteLine($"{Thread.CurrentThread.Name}: {c}");
            Thread.Sleep(300);
        }
    }
}
```

Рисунок 1.1 – Код програми

```
NumbersThread: 26
NumbersThread: 27
LettersThread: S
NumbersThread: 28
LettersThread: T
NumbersThread: 29
NumbersThread: 30
LettersThread: U
NumbersThread: 31
LettersThread: V
NumbersThread: 32
NumbersThread: 33
LettersThread: W
NumbersThread: 34
LettersThread: X
NumbersThread: 35
NumbersThread: 36
LettersThread: Y
NumbersThread: 37
LettersThread: Z
NumbersThread: 38
NumbersThread: 39
NumbersThread: 40
Thread execution has completed.
```

Рисунок 1.2 – Результат виконання роботи

**Питання для звіту**: поясніть, як працює багатозадачність у межах одного процесу.

У межах однієї запущеної програми ми можемо створити кілька паралельних гілок виконання (потоків), які розв'язують різні задачі. Процесор дуже швидко перемикається між ними. Якщо якась частина програми змушена простоювати (чекає на дані або таймер), операційна система віддає процесорний час іншому потоку цієї ж програми, що робить її роботу набагато ефективнішою.

**Завдання 2:** Створіть програму, яка запускає три потоки: Два пріоритетні (звичайні). Один фоновий. Організуйте так, щоб фоновий потік виконував нескінченний цикл з повідомленням у консоль.

```
using System;
using System.Threading;

class Program
{
    static void Main()
    {
        // First foreground thread
        Thread thread1 = new Thread(DoWork);
        thread1.Name = "Thread 1";

        // Second foreground thread
```

```
        Thread thread2 = new Thread(DoWork);
        thread2.Name = "Thread 2";

        // Background thread
        Thread backgroundThread = new Thread(BackgroundWork);
        backgroundThread.Name = "Background Thread";
        backgroundThread.IsBackground = true;

        // Start threads
        thread1.Start();
        thread2.Start();
        backgroundThread.Start();

        //thread1.Join();
        //thread2.Join();

        Console.WriteLine("Main thread has finished execution.");
    }

    static void DoWork()
    {
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} is working...
{i}");
            Thread.Sleep(500);
        }

        Console.WriteLine($"{Thread.CurrentThread.Name} has finished
work.");
    }

    static void BackgroundWork()
    {
        while (true)
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} is running in
the background...");
            Thread.Sleep(1000);
        }
    }
}
```

Рисунок 1.3 – Код програми

```
Main thread has finished execution.
Background Thread is running in the background...
Thread 1 is working... 1
Thread 2 is working... 1
Thread 2 is working... 2
Thread 1 is working... 2
Background Thread is running in the background...
Thread 1 is working... 3
Thread 2 is working... 3
Thread 2 is working... 4
Thread 1 is working... 4
Background Thread is running in the background...
Thread 1 is working... 5
Thread 2 is working... 5
Thread 2 has finished work.
Thread 1 has finished work.
```

Рисунок 1.4 – Результат роботи програми

Щоб програма працювала нескінченно потрібно закоментувати backgroundThread.IsBackground = true;

```
Main thread has finished execution.
Thread 1 is working... 1
Thread 2 is working... 1
Background Thread is running in the background...
Thread 2 is working... 2
Thread 1 is working... 2
Background Thread is running in the background...
Thread 1 is working... 3
Thread 2 is working... 3
Thread 1 is working... 4
Thread 2 is working... 4
Background Thread is running in the background...
Thread 2 is working... 5
Thread 1 is working... 5
Thread 2 has finished work.
Thread 1 has finished work.
Background Thread is running in the background...
Background Thread is running in the background...
Background Thread is running in the background...
Background Thread is running in the background...
```

Рисунок 1.5 – Результат роботи зміненої програми

**Питання для звіту:** що відбудеться з фоновим потоком після завершення пріоритетних потоків?

Фоновий потік не підтримує "життя" програми. Програма працює лише доти, доки працює хоча б один пріоритетний потік. Щойно останній пріоритетний потік фінішує, фоновий потік

автоматично переривається і знищується системою разом із завершенням усього процесу.

**Завдання 3:** створити багатопотокову програму, задати різні пріоритети для потоків. Кожен потік виконує підрахунок до 100 млн. Після завершення роботи програми визначте кількість ітерацій, виконаних кожним потоком, та розрахуйте відсотковий розподіл процесорного часу.

```
using System;
using System.Threading;
using System.Diagnostics;
using System.Collections.Generic;

class Program
{
    const long MaxCount = 100_000_000;
    static List<ThreadResult> results = new List<ThreadResult>();

    static void Main()
    {
        ThreadPriority[] priorities =
        [
            ThreadPriority.Normal,
            ThreadPriority.AboveNormal,
            ThreadPriority.Highest
        ];

        List<Thread> threads = new List<Thread>();

        for (int i = 0; i < priorities.Length; i++)
        {

            Thread thread = new Thread(() => CountTask());
            thread.Name = $"Thread {i + 1}";
            thread.Priority = priorities[i];

            threads.Add(thread);
        }

        Stopwatch totalTime = Stopwatch.StartNew();

        foreach (var t in threads)
            t.Start();

        foreach (var t in threads)
            t.Join();

        totalTime.Stop();

        Console.WriteLine("\nExecution completed.\n");

        double totalMilliseconds = 0;
        foreach (var r in results)
            totalMilliseconds += r.ElapsedMilliseconds;

        foreach (var r in results)
        {
            double percent = (r.ElapsedMilliseconds / totalMilliseconds) *
100;
```

```
            Console.WriteLine($"{r.Name}");
            Console.WriteLine($"Priority: {r.Priority}");
            Console.WriteLine($"Iterations: {r.Iterations}");
            Console.WriteLine($"Time: {r.ElapsedMilliseconds} ms");
            Console.WriteLine($"CPU Share: {percent:F2}%\n");
        }
    }

    static void CountTask()
    {
        Stopwatch sw = Stopwatch.StartNew();

        long counter = 0;
        while (counter < MaxCount)
        {
            counter++;
        }

        sw.Stop();

        lock (results)
        {
            results.Add(new ThreadResult
            {
                Name = Thread.CurrentThread.Name,
                Priority = Thread.CurrentThread.Priority,
                Iterations = counter,
                ElapsedMilliseconds = sw.ElapsedMilliseconds
            });
        }
    }
}

class ThreadResult
{
    public string Name;
    public ThreadPriority Priority;
    public long Iterations;
    public long ElapsedMilliseconds;
}
```

Рисунок 1.6 – Код програми

```
Execution completed.

Thread 3
Priority: Highest
Iterations: 100000000
Time: 212 ms
CPU Share: 33,28%

Thread 1
Priority: Normal
Iterations: 100000000
Time: 212 ms
CPU Share: 33,28%

Thread 2
Priority: AboveNormal
Iterations: 100000000
Time: 213 ms
CPU Share: 33,44%
```

Рисунок 1.7 – Результат роботи коду

**Питання для звіту:** чи завжди високопріоритетний потік отримує найбільше часу ЦП? Які фактори можуть впливати на планування?

Високий пріоритет дає потоку перевагу, але не гарантує 100% часу. На розподіл впливає те, чим займається потік: якщо він зупиняється або чекає на якийсь ресурс, замість нього відразу починає виконуватися фоновий або низькопріоритетний потік. Також велику роль відіграють внутрішні алгоритми планування самої Windows (чи іншої ОС), які стежать за загальним навантаженням на систему.

**Завдання 4:** Високий рівень: Розробіть консольний застосунок, який:

- Дозволяє користувачу ввести кількість потоків та їхні пріоритети.
- Запускає кожен потік із власним завданням (наприклад, підрахунок до заданого числа).
- Виводить у реальному часі хід виконання кожного потоку (лічильник, стан).
- Після завершення – формує таблицю розподілу процесорного часу між потоками у %.

```
using System;
using System.Threading;
using System.Diagnostics;
using System.Collections.Generic;

class Program
{
```

```
static object consoleLock = new object();
static List<ThreadResult> results = new List<ThreadResult>();
static long maxCount;

static void Main()
{
    Console.Write("Enter number of threads: ");
    int threadCount = int.Parse(Console.ReadLine());

    Console.Write("Enter max count value (e.g. 100000000): ");
    maxCount = long.Parse(Console.ReadLine());

    List<Thread> threads = new List<Thread>();

    for (int i = 0; i < threadCount; i++)
    {
        Console.WriteLine($"\nSelect priority for Thread {i + 1}:");
        Console.WriteLine("1 - Lowest");
        Console.WriteLine("2 - BelowNormal");
        Console.WriteLine("3 - Normal");
        Console.WriteLine("4 - AboveNormal");
        Console.WriteLine("5 - Highest");

        int choice = int.Parse(Console.ReadLine());

        ThreadPriority priority = choice switch
        {
            1 => ThreadPriority.Lowest,
            2 => ThreadPriority.BelowNormal,
            4 => ThreadPriority.AboveNormal,
            5 => ThreadPriority.Highest,
            _ => ThreadPriority.Normal
        };

        Thread t = new Thread(CountTask);
        t.Name = $"Thread {i + 1}";
        t.Priority = priority;

        threads.Add(t);
    }

    Stopwatch totalTime = Stopwatch.StartNew();

    foreach (var t in threads)
        t.Start();

    foreach (var t in threads)
        t.Join();

    totalTime.Stop();

    PrintResults();
}

static void CountTask()
{
    Stopwatch sw = Stopwatch.StartNew();
    long counter = 0;

    while (counter < maxCount)
    {
        counter++;
```

```
            // Show progress every 10%
            if (counter % (maxCount / 10) == 0)
            {
                lock (consoleLock)
                {
                    Console.WriteLine($"{Thread.CurrentThread.Name} | " +
                                      $"Priority:
{Thread.CurrentThread.Priority} | " +
                                      $"Progress: {(counter * 100) /
maxCount}%");
                }
            }
        }

        sw.Stop();

        lock (results)
        {
            results.Add(new ThreadResult
            {
                Name = Thread.CurrentThread.Name,
                Priority = Thread.CurrentThread.Priority,
                TimeMs = sw.ElapsedMilliseconds
            });
        }
    }

    static void PrintResults()
    {
        double totalTime = 0;
        foreach (var r in results)
            totalTime += r.TimeMs;

        Console.WriteLine("Name\t\tPriority\tTime(ms)\tCPU %");
        Console.WriteLine("----------------------------------------------
-------");

        foreach (var r in results)
        {
            double percent = (r.TimeMs / totalTime) * 100;


Console.WriteLine($"{r.Name}\t{r.Priority}\t{r.TimeMs}\t\t{percent:F2}%");
        }
    }
}

class ThreadResult
{
    public string Name;
    public ThreadPriority Priority;
    public long TimeMs;
}
```

Рисунок 1.8 – Код програми

```
Enter number of threads: 3
Enter max count value (e.g. 100000000): 9999999999

Select priority for Thread 1:
1 - Lowest
2 - BelowNormal
3 - Normal
4 - AboveNormal
5 - Highest
1

Select priority for Thread 2:
1 - Lowest
2 - BelowNormal
3 - Normal
4 - AboveNormal
5 - Highest
2

Select priority for Thread 3:
1 - Lowest
2 - BelowNormal
3 - Normal
4 - AboveNormal
5 - Highest
3
Thread 3 | Priority: Normal | Progress: 9%
Thread 2 | Priority: BelowNormal | Progress: 9%
Thread 1 | Priority: Lowest | Progress: 9%
```

Рисунок 1.9 – Результат роботи коду

**Висновок:** Під час виконання лабораторної роботи я навчився створювати багатопоточні програми та керувати пріоритетами потоків.